We'll illustrate these examples in figures 6.6 and 6.7, but first, let's take a look at the implementation of the search method, that we will then use as a reference while describing these examples.

Before delving into the examples, I'd like to quickly delve into the recursive definition of a string; a string s, in fact, can be defined as being either:

- The empty string, "";
- The concatenation of a character `c` and a string `s'`: `s=c+s'`, where `s'` is a string one character shorter than s, and can possibly be the empty string.

Note that, since in most programming languages single quote characters wouldn't be allowed in variable names, we use `tail`, in listing 6.2 and in the next ones, as a substitute name for `s'`, which denotes the tail of current string `s` in the figures.

In listing 6.2 we show a recursive implementation of the search method: at each call, we need to check if the first character in the substring searched matches an outgoing edge of current node, and then recursively search the tail of current string in the subtree referenced by that edge. Figures 6.6 and 6.7 shows the parallel between moving "right" in the string searched and traversing down the trie.

The same method can obviously be implemented using explicit loops; this implementation of the search method is likely suitable for compiler's tail-call optimization[12] but, as discussed in appendix E and chapter 3, if you are not comfortable with recursion or not sure that your compiler will